Introduction to Form Handling

1.Definition:

Form Handling in Angular is the process of collecting, managing, and processing user input through forms. Forms are used to accept information from users, such as name, email, password, and phone number.

Explanation:

1. **Collect User Input: Forms collect data entered by users in input fields.**
2. **Data Binding: Angular connects form fields with component data.**
3. **Validation: It checks whether the entered data is correct or not.**
4. **Submit Form: After entering the details, users can submit the form.**
5. **Error Handling: Angular can display error messages when users enter invalid data.**
6. **Process Data: The component processes the submitted data or sends it to a server.**

Types of Forms in Angular:

1. **Template-Driven Forms: These forms are mainly managed using HTML templates and Angular directives such as ngModel.**
2. **Reactive Forms: These forms are managed in the component class using classes such as FormControl and FormGroup.**

```
Example:
<form>
  <label>Name:</label>
  <input type="text" name="name">
  <button type="submit">Submit</button>
</form>
```

Form Validation

Definition:

Form Validation in Angular is the process of checking whether the data entered by the user in a form is correct and valid or not. It helps prevent users from submitting incorrect or incomplete information.

Explanation

Form validation is important for maintaining the accuracy of user data. Angular provides different validation methods to check form inputs.

1. **Required Validation: It checks whether a field is empty or filled.**
2. **Email Validation: It checks whether the entered email address has a valid format.**
3. **Minimum Length: It checks whether the input contains the minimum required number of characters.**
4. **Maximum Length: It checks whether the input exceeds the maximum allowed number of characters.**
5. **Pattern Validation: It checks whether the input matches a specified pattern, such as a phone number.**
6. **Error Messages: Angular displays error messages when the user enters invalid data.**

Example

```
<form #myForm="ngForm">
  <input
    type="text"
    name="username"
    ngModel
    required
    #user="ngModel">
  <div *ngIf="user.invalid && user.touched">
    Username is required.
  </div>
  <button type="submit">Submit</button>
</form>
```

Explanation of the example:

- **ngForm manages the form.**
- **ngModel connects the input field with Angular.**
- **required makes the field compulsory.**
- **user.invalid checks whether the field is invalid.**
- **user.touched checks whether the user has interacted with the field.**
- **The error message appears when the field is invalid and has been touched.**

Advantages of Form Validation

1. **It prevents incomplete data from being submitted.**
2. **It improves data accuracy.**
3. **It provides helpful error messages.**
4. **It improves the user experience.**
5. **It reduces incorrect data entry.**

ng-minlength

1.Definition:

ng-minlength is a directive used to specify the minimum number of characters allowed in an input field. It helps ensure that the user enters a minimum length of data.

Explanation:

1. **It checks the minimum length of the input value.**
2. **If the input contains fewer characters than required, it becomes invalid.**
3. **It is used for fields such as username, password, and other text inputs.**
4. **It can be used to display an error message when the input is too short.**

```
Example:
<form name="myForm">
  <input type="text"
         name="username"
         ng-model="username"
         ng-minlength="5">
  <span ng-show="myForm.username.$error.minlength">
    Minimum 5 characters required.
  </span>
</form>
```

Explanation of the example**:**

- **ng-model binds the input value to the variable username.**
- **ng-minlength="5" requires at least 5 characters.**
- **$error.minlength checks whether the input is shorter than 5 characters.**
- **ng-show displays the error message when the minimum length rule is violated.**

---

ngmaxlength

Definition:

ng-maxlength is a directive used to specify the maximum number of characters allowed in an input field. It checks that the user does not enter more characters than the specified limit.

Explanation

1. **It sets the maximum length of an input value.**
2. **If the input contains more characters than allowed, the field becomes invalid.**
3. **It is useful for fields such as usernames, passwords, and descriptions.**
4. **It can display an error message when the maximum length is exceeded.**

Advantages 1. It prevents excessively long input values. 2.It helps maintain data consistency.

3. **It improves form validation.**
4. **It provides feedback to users.**

```
Example
<form name="myForm">
  <input type="text"
         name="username"
         ng-model="username"
         ng-maxlength="10">
  <span ng-show="myForm.username.$error.maxlength">
    Maximum 10 characters allowed. </span>
</form>
```

Explanation of the Example

- **ng-model binds the input value to the username variable.**
- **ng-maxlength="10" sets the maximum length to 10 characters.**
- **$error.maxlength checks whether the input exceeds the limit.**
- **ng-show displays the error message when the input is too long.**

ng-pattern

Definition

ng-pattern is a directive in AngularJS used to validate user input against a specified pattern or regular expression. It checks whether the entered data matches the required format.

Explanation

1. **Pattern Matching: It checks whether the input matches a given pattern.**
2. **Validation: If the input does not match the pattern, the field becomes invalid.**
3. **Email and Phone Numbers: It can validate formats such as phone numbers, usernames, and other text inputs.**
4. **Error Messages: It can display an error message when the input does not match the required pattern.**

Example

```
<form name="myForm">
  <input type="text"
         name="mobile"
         ng-model="mobile"
         ng-pattern="/^[0-9]{10}$/">
  <span ng-show="myForm.mobile.$error.pattern">
    Enter a valid 10-digit mobile number.
  </span>
</form>
```

Explanation of the Example

- **ng-model binds the input value to the mobile variable.**
- **ng-pattern checks whether the input contains exactly 10 digits.**
- **$error.pattern checks whether the entered value fails the pattern validation.**
- **ng-show displays the error message when the input is invalid.**

Advantages

1. **It validates input formats.**
2. **It reduces incorrect data entry.**
3. **It helps maintain data accuracy.**
4. **It provides useful error messages.**

ng-required

Definition:

ng-required is a directive in AngularJS used to make an input field compulsory. It ensures that the user enters a value before submitting the form.

Explanation

1. **Required Field: It makes an input field mandatory.**
2. **Validation: If the field is empty, it becomes invalid.**
3. **Dynamic Requirement: It can make a field required based on a condition.**
4. **Error Message: It can display an error message when the user leaves the required field empty.**

Example

```
<form name="myForm">
  <input type="text"
         name="username"
         ng-model="username"
         ng-required="true">
  <span ng-show="myForm.username.$error.required">
    Username is required.
  </span>
  <button type="submit">Submit</button>
</form>
```

Explanation **of the Example**

- **ng-model binds the input value to the username variable.**
- **ng-required="true" makes the field compulsory.**
- **$error.required checks whether the required field is empty.**
- **ng-show displays the error message when the field is empty.**

Advantages

1. **It prevents empty values in required fields002E**
2. **It improves data accuracy.**
3. **It helps users complete forms correctly.**
4. **It supports conditional validation.**

Submitting Forms

In Definition:

Submitting Forms is the process of sending the data entered by a user for further processing. In AngularJS, the ng-submit directive is used to execute a function when a form is submitted.

Explanation

1. **Collect Data: The form collects information entered by the user.**
2. **Form Validation: AngularJS checks whether the entered data is valid.**
3. **Submit Form: The user clicks the Submit button to submit the form.**
4. **Execute Function: The ng-submit directive calls a function when the form is submitted.**
5. **Process Data: The function processes the submitted information or sends it to a server.**

Example: Submitting Forms in AngularJS

```
<form ng-submit="submitForm()">
  <input type="text" ng-model="name" required>
  <button type="submit">Submit</button>
</form>
$scope.submitForm = function() {
  alert("Form Submitted!");
};
```

Explanation:

- **ng-submit calls the submitForm() function.**
- **ng-model stores the entered name.**
- **required makes the field compulsory.**
- **alert() displays a message after submission.**

Advantages

1. **It makes form submission easy.**
2. **It supports form validation.**
3. **It allows data to be processed using controller functions.**
4. **It provides feedback after submission.**

Event Handling with Forms.

1. Definition

Event Handling with Forms is a process of responding to user actions in a form. AngularJS provides different directives to handle events such as clicking, typing, changing input values, and submitting forms.

Common Event Directives:

1. **ng-click – Executes a function when a button is clicked.**
2. **ng-change – Executes a function when an input value changes.**
3. **ng-submit – Executes a function when a form is submitted.**
4. **ng-focus – Executes an action when an input field receives focus.**
5. **ng-blur – Executes an action when an input field loses focus.**

Example:

```
<form ng-submit="submitForm()">
  <input type="text" ng-model="name"
         ng-change="update()">
  <button type="submit">Submit</button>
</form>
$scope.update = function() {
  alert("Value Changed!");  };
$scope.submitForm = function() {
  alert("Form Submitted!"); };
```

Explanation:

- **ng-model stores the entered value in the name variable.**
- **ng-change calls the update() function when the value changes.**
- **ng-submit calls the submitForm() function when the form is submitted.**
- **alert() displays a message on the screen.**

Advantages:

1. **Makes forms interactive.**
2. **Responds to user actions.**
3. **Helps validate and process user input.**
4. **Improves the user experience.**